

ZAAWANSOWANY FRAMEWORK PHP

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Rozpoczęcie	1
Uwaga	2
Ponowne klonowanie repozytorium	2
Pobranie i uruchomienie composera	2
Instalacja Symfony.....	2
Wstępny commit projektu do GIT	3
Baza danych.....	3
Tworzenie CRUD.....	6
Bootstrap.....	11
Własny responsywny wygląd	13
Commit projektu do GIT.....	17
Podsumowanie.....	17

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- instalacja i konfiguracja frameworka Symfony;
- wykorzystanie narzędzia composer;
- połączenia z bazą danych;
- tworzenie systemów webowych na podstawie istniejących prototypów.

W praktycznym wymiarze uczestnicy wykorzystają Symfony Framework do odtworzenia funkcjonalności naiwnego frameworka z poprzedniego laboratorium.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie zasad działania repozytorium GIT: `clone`, `pull`, `push`, `commit`, `fetch`, `merge`, `init`. Powtórzenie informacji o aktualnych wersjach Symfony. Powtórzenie sposobów instalacji Symfony (skrypt `symfony` lub `composer`). Powtórzenie wersji frameworka – `skeleton` i `website-skeleton`. Powtórzenie działania mapowania obiektowo-relacyjnego w Symfony oraz komend `make:entity` i `make:crud`.

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub Ctrl+A -> F9.

PONOWNE KLONOWANIE REPOZYTORIUM

W związku z problemami wydajnościowymi w odczycie plików na macierzy dyskowej I:\, to laboratorium musimy wykonać na pulpicie. Jest to **bardzo zła** praktyka, ale niestety administratorom nie udało się znaleźć innego rozwiązania ani permanentnego obejścia. Wyłącznie jeśli korzystasz z pulpitu zdalnego do realizacji zadań, ponownie sklonuj swoje repozytorium na pulpit, tj. do katalogu analogicznego do C:\Users\akarczmarczyk\Desktop\ptw.

Na tym etapie warto w ustawieniach tej karty pracy zmienić wartość niestandardowej właściwości labmdir oraz labdir, po czym odświeżyć wszystkie pola w dokumencie Ctrl+A, F9.

POBRANIE I URUCHOMIENIE COMPOSERA

Wejdź na stronę <https://getcomposer.org/download/>. Przewiń w dół, do sekcji Manual Download i kliknij prawym przyciskiem myszy na wersję 2.9.7. Zapisz jako C:\Users\akarczmarczyk\Desktop\ptw\composer.phar. Przejdź terminalem do katalogu, do którego pobrano composer i wykonaj polecenie:

```
php composer.phar --version
```

W efekcie powinniśmy dostać informację o wersji composera i PHP, przykładowo:

```
PS C:\Users\akarczmarczyk\Desktop\ptw> php .\composer.phar --version
Composer version 2.0.18 2026-04-14 13:31:52
PHP version 8.5.5 (I:\php-8.5.5-win32-vs17-x64\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
```

INSTALACJA SYMFONY

Wejdź terminalem do katalogu C:\Users\akarczmarczyk\Desktop\ptw. Zainstaluj Symfony 7.4 w wersji website-skeleton:

```
cd C:\Users\akarczmarczyk\Desktop\ptw
php composer.phar create-project symfony/skeleton:"7.4.x" lab-h
cd lab-h
php ..\composer.phar require webapp
```

Na pytanie o instalację receptury kontenerów Docker odpowiedz **przecząco**.

Ostatni krok trochę potrwa. To idealny moment, żeby poczytać dokumentację dostępną tutaj: <https://symfony.com/doc/current/setup.html>.

Po zakończeniu instalacji, wejdź do katalogu projektu i uruchom wbudowany serwer PHP:

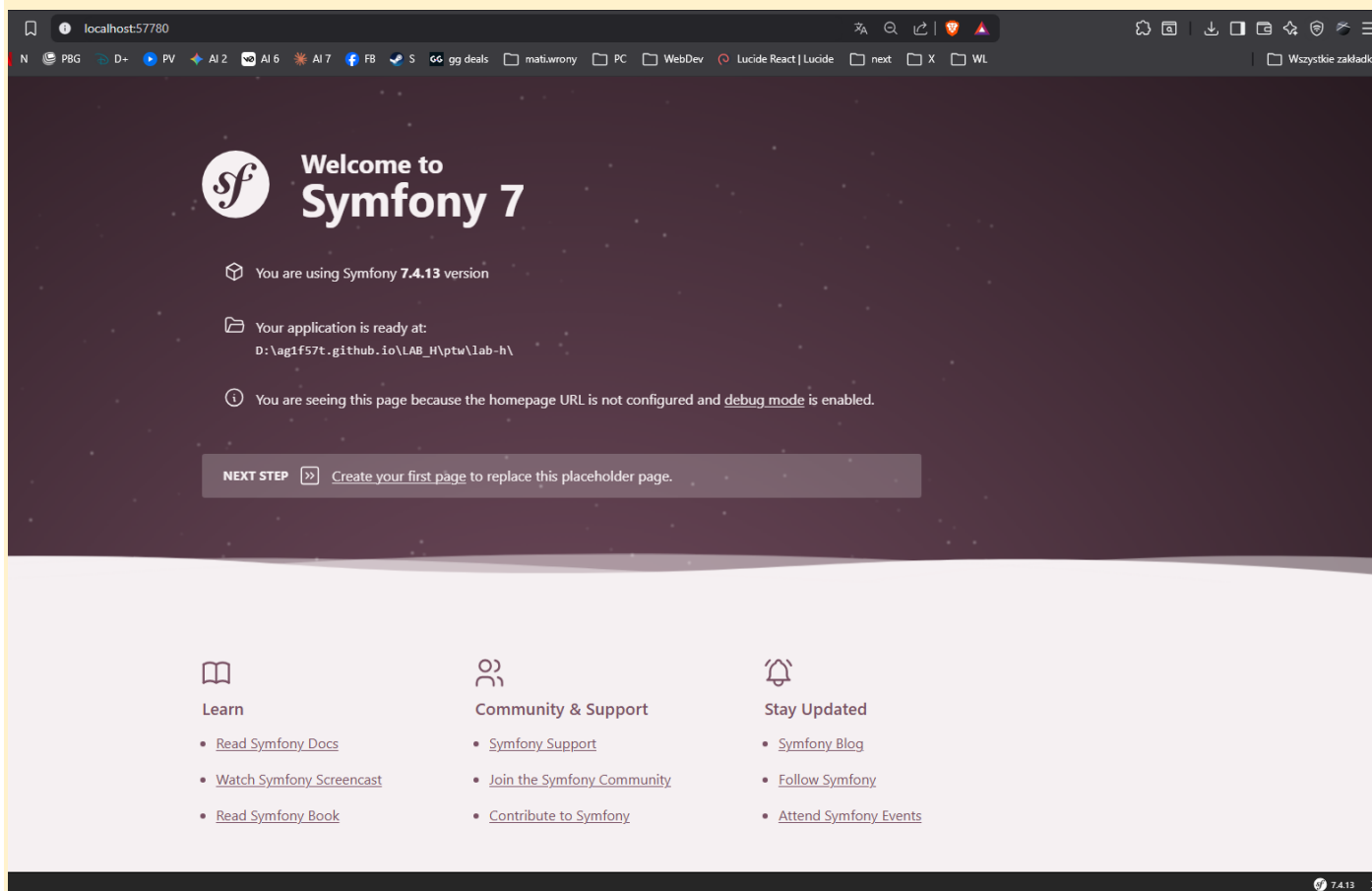
```
cd C:\Users\akarczmarczyk\Desktop\ptw\lab-h  
php -S localhost:57780 -t .\public\
```

Powyżej wyróżniony został numer indeksu. W przypadku powodzenia efekt powinien być zbliżony do poniższego:

```
PS C:\Users\akarczmarczyk\Desktop\ptw\lab-h> php -S localhost:3346 -t .\public\  
[Fri Apr 24 00:31:58 2026] PHP 8.5.5 Development Server (http://localhost:3346) started
```

Otwórz ulubioną przeglądarkę internetową i wejdź pod adres `http://localhost:57780`.

Zamień poniższy zrzut ekranu na Twój odpowiednik. Upewnij się, że widoczne i poprawne są wszystkie zaznaczone fragmenty – adres URL i port, wersja Symfony 7.x, ścieżka na dysku:



Punkty:	0	1
---------	---	---

WSTĘPNY COMMIT PROJEKTU DO GIT

To dobry moment, żeby zacommitować projekt do repozytorium. Upewnij się, że w `.gitignore` globalnie zablokowane są katalogi `.idea`. Dodaj do repozytorium cały katalog `lab-h`, zacommituj i pushnij zmiany.

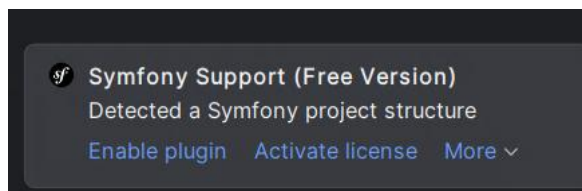
BAZA DANYCH

W tej sekcji skonfigurujemy połączenie z bazą danych SQLite i zbudujemy encję.

Podaj nazwę swojego modelu z poprzedniego laboratorium: `Recipe` (zamiast edytować ręcznie, zmień niestandardową właściwość `encja` w tym dokumencie, żeby w dalszych instrukcjach było łatwiej).

Rozpocznij od uruchomienia katalogu projektu `lab-h` w PhpStorm. To ważne, żeby katalog `lab-h` był rootem projektu, a nie podkatalogiem – będzie o wiele wygodniej!

Zainstaluj i włącz w ustawieniach PhpStorm plugin `Symfony Plugin`. Konieczny będzie restart PhpStorm. Włącz plugin dla bieżącego projektu (ponowny restart):



W pliku `.env` zmień bazę danych na SQLite:

```
# .env
#...
DATABASE_URL="sqlite:///kernel.project_dir%/var/data_%kernel.environment%.db"
```

Ten wpis oznacza, że aplikacja będzie korzystała z bazy danych SQLite umieszczonej w pliku `C:\Users\akarczmarczyk\Desktop\ptw\lab-h\var\data_dev.db`.

Uwaga! Plik `.env` jest analizowany przez Symfony od góry do dołu. W poprzednich edycjach kursu studenci często mimo odkomentowania linijki z SQLite, zapominali zakomentować linijkę z domyślnym PostgreSQL. To skutkowało nadpisaniem zmiennej środowiskowej `DATABASE_URL`, a w efekcie ignorowaniem bazy danych SQLite. Ponieważ w `php.ini` nie włączyliśmy rozszerzenia PDO dla PostgreSQL, w takim przypadku wizualnie w przeglądarce wyskakuje błąd o braku sterownika.

Otwórz nowy terminal i przejdź do katalogu projektu. Wykonaj polecenia, w celu utworzenia encji `Recipe`. Przykład wykorzystania przedstawiony został na wykładzie. Ustaw pola zgodne z Twoim modelem na poprzednim laboratorium, przykładowo:

```

PS C:\Users\akarczmarczyk\Desktop\ptw\lab-h> php .\bin\console make:entity

Class name of the entity to create or update (e.g. VictoriousPuppy):
> Post

Add the ability to broadcast entity updates using Symfony UX Turbo? (yes/no) [no]
]:
>

created: src/Entity/Post.php
created: src/Repository/PostRepository.php

Entity generated! Now let's add some fields!
You can always add more fields later manually or by re-running this command.
New property name (press <return> to stop adding fields):
> subject

Field type (enter ? to see all types) [string]:
>

Field length [255]:
>

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Post.php

Add another property? Enter the property name (or press <return> to stop adding
fields):
> description

Field type (enter ? to see all types) [string]:
> text

Can this field be null in the database (nullable) (yes/no) [no]:
>

updated: src/Entity/Post.php

Add another property? Enter the property name (or press <return> to stop adding
fields):
>

Success!

```

Na tym etapie model danych nie został jeszcze naniesiony na bazę danych. Wykonaj komendy:

```
php bin\console doctrine:schema:update --dump-sql
```

```
php bin\console doctrine:schema:update --dump-sql --force
```

Czym różni się --dump-sql od --force? **Nie używaj LLM.**

Bez --force nie robi ale pokaze, a z --force zrobi

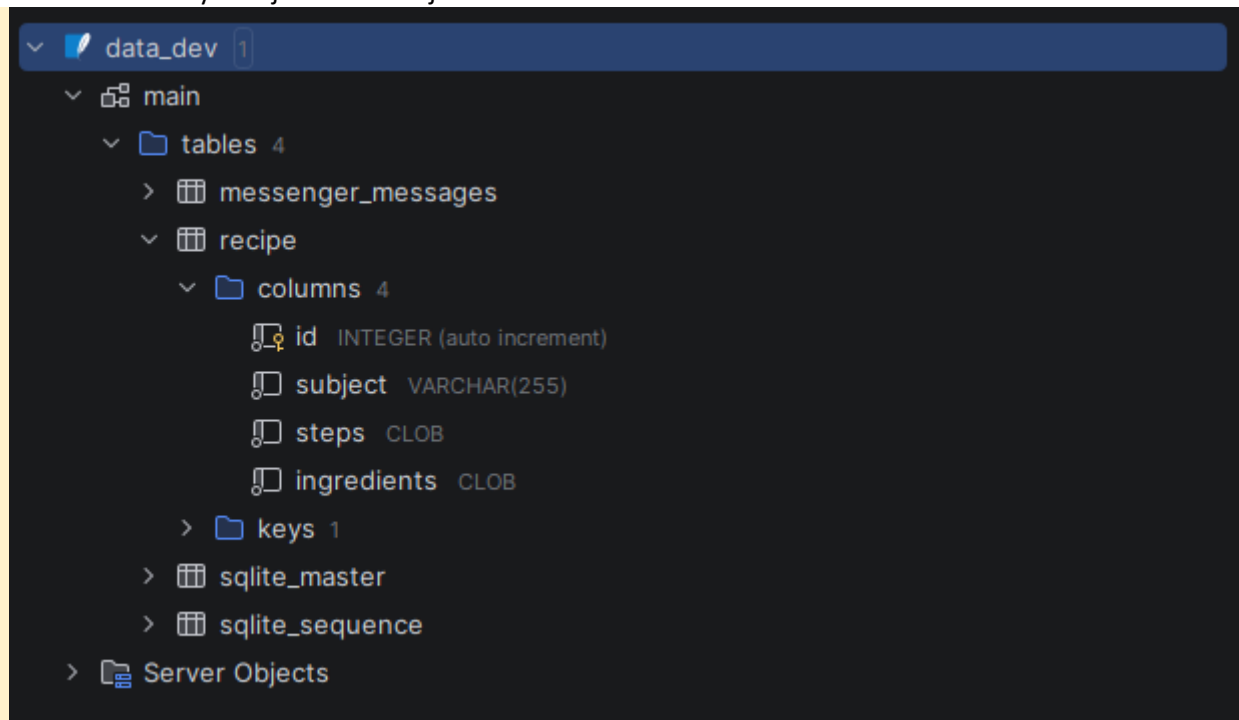
Umieść zrzut ekranu kodu SQL, który został wygenerowany:

```

PS D:\ag1f57t.github.io\LAB_H\ptw\lab-h> php bin\console doctrine:schema:update --dump-sql
CREATE TABLE recipe (id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, subject VARCHAR(255) NOT NULL, steps CLOB NOT NULL,
ingredients CLOB NOT NULL);
CREATE TABLE messenger_messages (id INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, body CLOB NOT NULL, headers CLOB NOT NULL,
queue_name VARCHAR(190) NOT NULL, created_at DATETIME NOT NULL, available_at DATETIME NOT NULL, delivered_at DATETIME
DEFAULT NULL);
CREATE INDEX IDX_75EA56E0FB7336F0E3BD61CE16BA31DBBF396750 ON messenger_messages (queue_name, available_at, delivered_at,
id);

```

Wykorzystaj PhpStorm do połączenia się z bazą danych w pliku C:\Users\akarczmarczyk\Desktop\ptw\lab-h\var\data_dev.db. Umieść poniżej zrzut ekranu drzewa tabel/kolumn:



Punkty:

0

1

TWORZENIE CRUD

W kolejnym etapie wykorzystamy komendę `make:crud` do wygenerowania zestawu **Create, Read, Update, Delete** encji `Recipe`. Nawiguj terminalem do katalogu projektu i wykonaj polecenie:

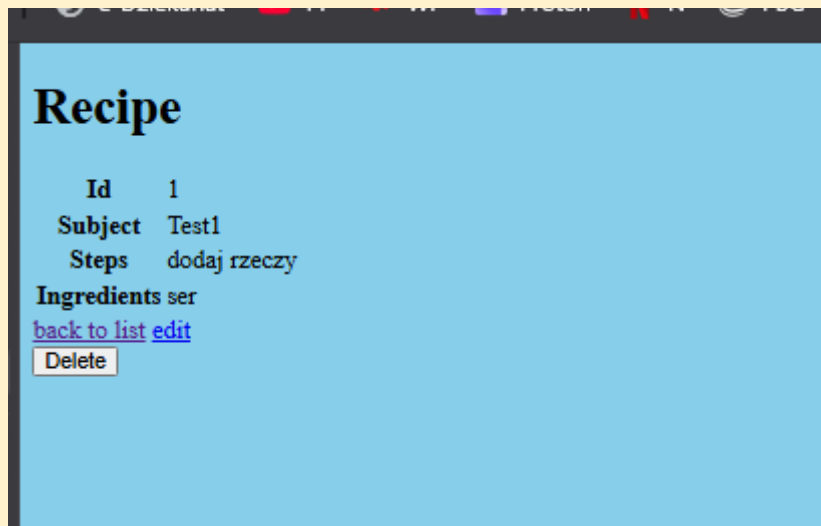
```
php .\bin\console make:crud
```

Przejdź przez kreator, a następnie wejdź na stronę <http://localhost:57780/Recipe/>. Powinna pokazać się gotowa aplikacja z możliwością przeglądania, dodawania, edycji i kasowania encji `Recipe`.

Umieść zrzut ekranu strony index z listą Twoich encji. Upewnij się, że to ten sam model co w poprzednim laboratorium:



Zrzut ekranu strony szczegółów encji:



Zrzut ekranu strony edycji encji:



Wstaw zrzuty ekranu kodów akcji kontrolera dla wszystkich powyższych ekranów:

```

#[Route(name: 'app_recipe_index', methods: ['GET'])]
public function index(RecipeRepository $recipeRepository): Response
{
    return $this->render( view: 'recipe/index', [
        'recipes' => $recipeRepository->findAll(),
    ]);
}

```

no usages

```

45      #[Route('/{id}', name: 'app_recipe_show', methods: ['GET'])]
46      public function show(Recipe $recipe): Response
47      {
48          return $this->render( view: 'recipe/show', [
49              'recipe' => $recipe,
50          ]);
51      }
52

```

no usages

```

53      #[Route('/{id}/edit', name: 'app_recipe_edit', methods: ['GET', 'POST'])]
54      public function edit(Request $request, Recipe $recipe, EntityManagerInterface $entityManager): Response
55      {
56          $form = $this->createForm( type: RecipeType::class, $recipe);
57          $form->handleRequest($request);
58
59          if ($form->isSubmitted() && $form->isValid()) {
60              $entityManager->flush();
61
62              return $this->redirectToRoute( route: '/recipe', [], status: Response::HTTP_SEE_OTHER);
63          }
64
65          return $this->render( view: 'recipe/edit', [
66              'recipe' => $recipe,
67              'form' => $form,
68          ]);
69      }
70

```

Wstaw zrzuty ekranu kodów szablonów TWIG dla wszystkich powyższych ekranów:


```

4
5 {% block body %}
6     <h1>Recipe index</h1>
7
8     <table class="table">
9         <thead>
10             <tr>
11                 <th>Id</th>
12                 <th>Subject</th>
13                 <th>Steps</th>
14                 <th>Ingredients</th>
15                 <th>actions</th>
16             </tr>
17         </thead>
18         <tbody>
19             {% for recipe in recipes %}
20                 <tr>
21                     <td>{{ recipe.id }}</td>
22                     <td>{{ recipe.subject }}</td>
23                     <td>{{ recipe.steps }}</td>
24                     <td>{{ recipe.ingredients }}</td>
25                     <td>
26                         <a href="{{ path('/recipe/{id}', {'id': recipe.id}) }}">show</a>
27                         <a href="{{ path('/recipe/{id}/edit', {'id': recipe.id}) }}">edit</a>
28                     </td>
29                 </tr>
30             {% else %}
31                 <tr>
32                     <td colspan="5">no records found</td>
33                 </tr>
34             {% endfor %}
35         </tbody>
36     </table>
37
38     <a href="{{ path('/recipe/new') }}">Create new</a>
39 {% endblock %}
40

```

```
.env RecipeController.php index.html.twig show.html.twig x
1 {% extends 'base.html.twig' %}
2
3 {% block title %}Recipe{% endblock %}
4
5 {% block body %}
6     <h1>Recipe</h1>
7
8     <table class="table">
9         <tbody>
10             <tr>
11                 <th>Id</th>
12                 <td>{{ recipe.id }}</td>
13             </tr>
14             <tr>
15                 <th>Subject</th>
16                 <td>{{ recipe.subject }}</td>
17             </tr>
18             <tr>
19                 <th>Steps</th>
20                 <td>{{ recipe.steps }}</td>
21             </tr>
22             <tr>
23                 <th>Ingredients</th>
24                 <td>{{ recipe.Ingredients }}</td>
25             </tr>
26         </tbody>
27     </table>
28
29     <a href="{{ path('/recipe') }}">back to list</a>
30
31     <a href="{{ path('/recipe/{id}/edit', {'id': recipe.id}) }}">edit</a>
32
33     {{ include('recipe/_delete_form.html.twig') }}
34 {% endblock %}
35
```

```

Variables (3)
1  {% extends 'base.html.twig' %}
2
3  {% block title %}Edit Recipe{% endblock %}
4
5  {% block body %}
6      <h1>Edit Recipe</h1>
7
8      {{ include('recipe/_form.html.twig', {'button_label': 'Update'}) }}
9
10     <a href="{{ path('/recipe') }}">back to list</a>
11
12     {{ include('recipe/_delete_form.html.twig') }}
13 {% endblock %}
14

```

Punkty:	0	1
---------	---	---

BOOTSTRAP

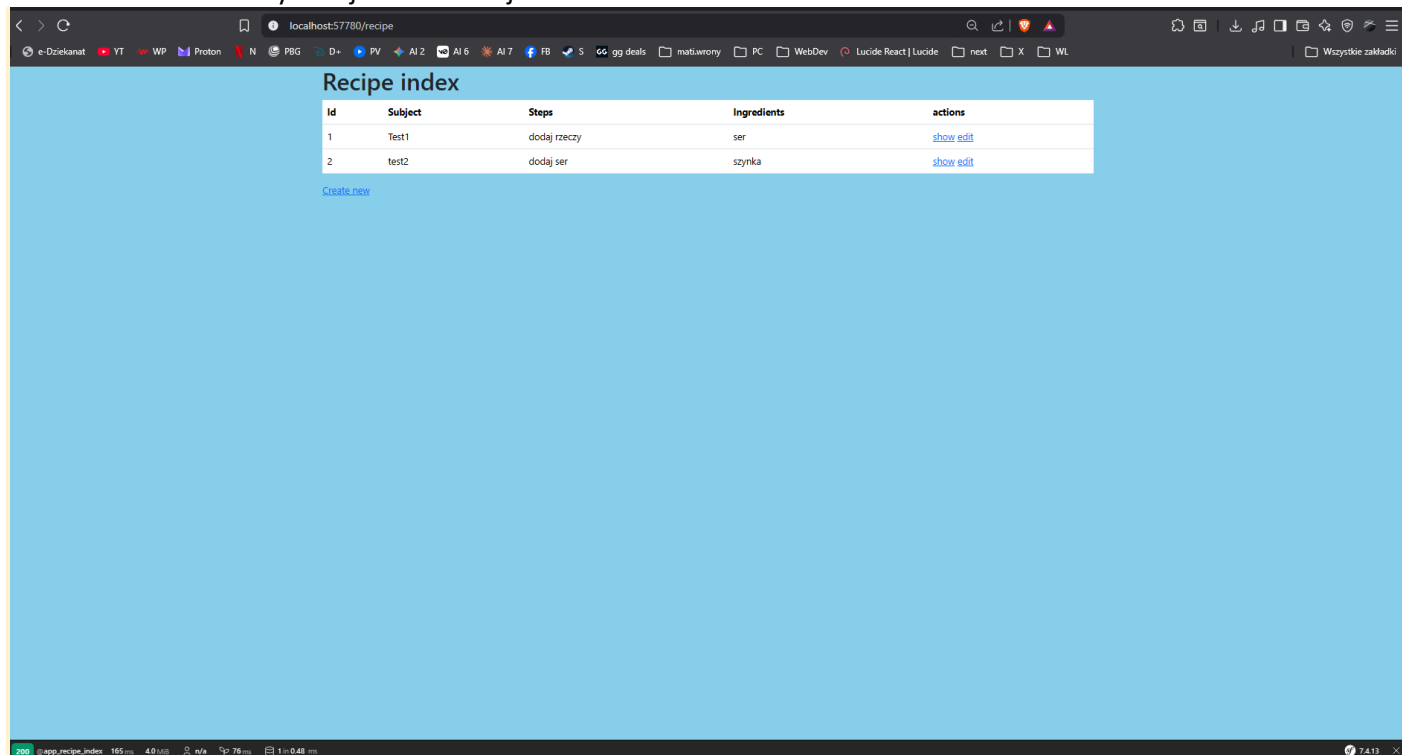
Korzystając z notatek z wykładu oraz poniższych adresów, podłącz do aplikacji i jej formularzy bibliotekę Bootstrap:

- <https://symfony.com/doc/current/form/bootstrap5.html>
- <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

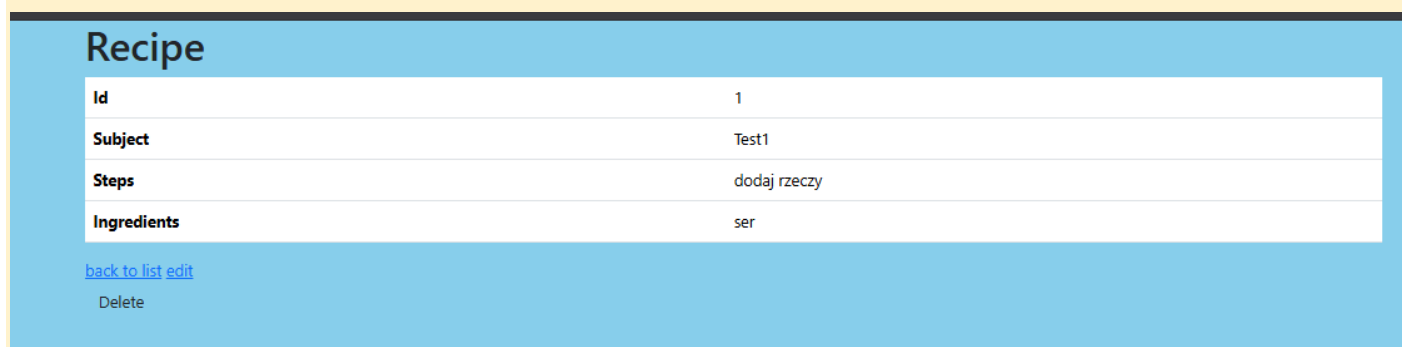
W szczególności, edytuj pliki:

- base.html.twig
- twig.yaml
- app.css
- templates/Recipe/*.twig

Umieść zrzut ekranu strony index z listą Twoich encji. Upewnij się, że zastosowano kontener z klasą `container` albo `container-fluid`:



Zrzut ekranu strony szczegółów encji:



Zrzut ekranu strony edycji encji:



Punkty:	0	1
---------	---	---

WŁASNY RESPANSYWNY WYGLĄD

Na koniec poświęć trochę czasu, żeby nadać witrynie unikalny, własny wygląd. Zastosuj komponenty Bootstrap oraz wykorzystaj własne nazwy klas oraz plik app.css do własnego formatowania. Upewnij się, że jest co najmniej header i footer, oraz że strona jest responsywna.

Przydatne linki:

- <https://getbootstrap.com/docs/5.3/components/navbar/>
- <https://getbootstrap.com/docs/5.3/helpers/color-background/>
- <https://getbootstrap.com/docs/5.3/content/tables/>
- <https://getbootstrap.com/docs/5.3/content/typography/>

Umieść zrzut ekranu strony index z listą Twoich encji. Szeroki i wąski ekran.

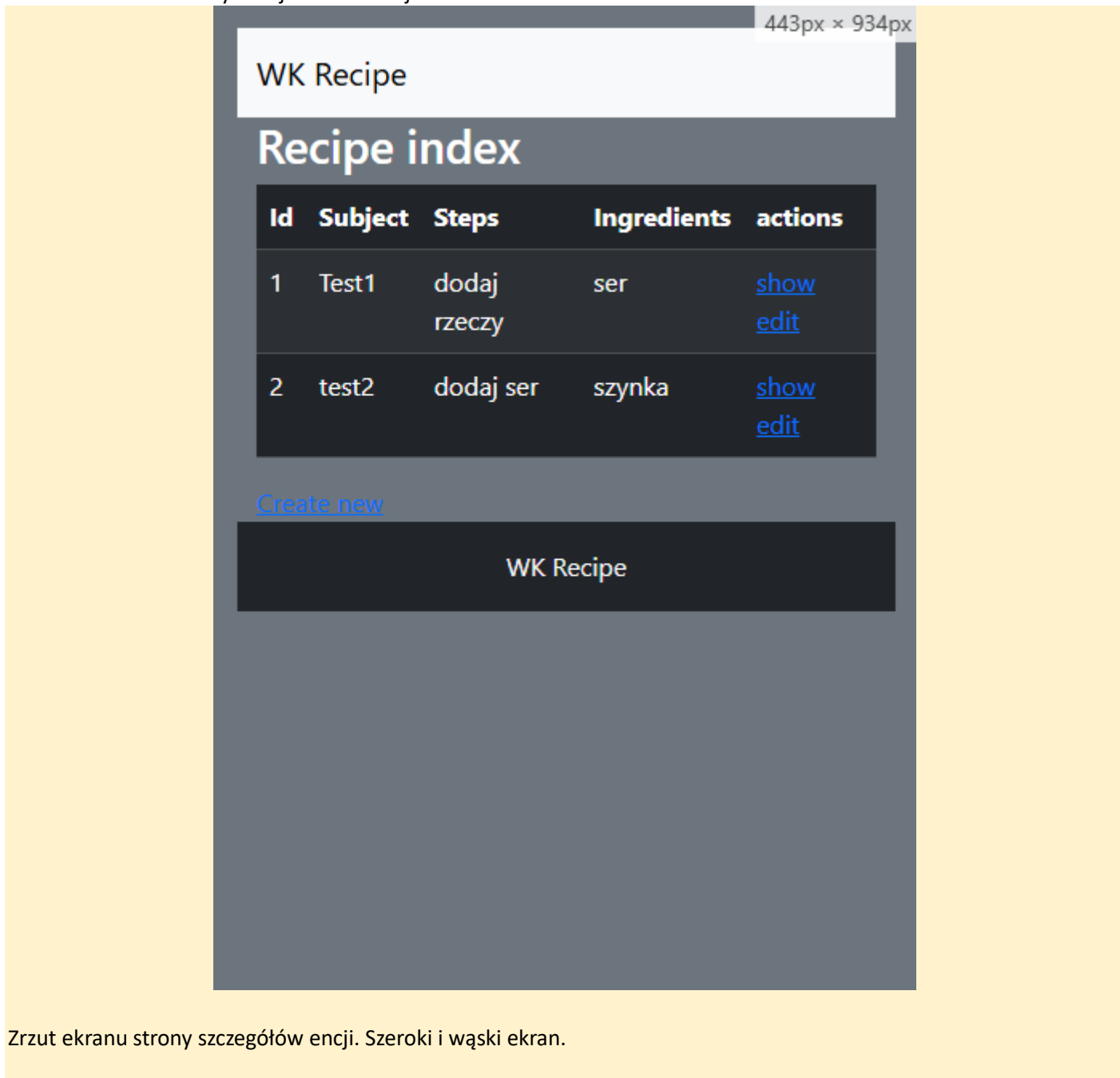
WK Recipe

Recipe index

Id	Subject	Steps	Ingredients	actions
1	Test1	dodaj rzeczy	ser	show edit
2	test2	dodaj ser	szynka	show edit

[Create new](#)

WK Recipe



Zrzut ekranu strony szczegółów encji. Szeroki i wąski ekran.

WK Recipe

Recipe

Id	2
Subject	test2
Steps	dodaj ser
Ingredients	szynka

[back to list edit](#)
Delete

WK Recipe

WK Recipe

Recipe

Id	2
Subject	test2
Steps	dodaj ser
Ingredients	szynka

[back to list edit](#)
Delete

WK Recipe

Zrzut ekranu strony edycji encji. Szeroki i wąski ekran:

WK Recipe

Edit Recipe

Subject
test2

Steps
dodaj ser

Ingredients
szynka

[Update](#)
[back to list](#)
Delete

WK Recipe

320px × 934px

WK Recipe

Edit Recipe

Subject

test2

Steps

dodaj ser

Ingredients

szynka

Update

[back to list](#)

Delete

WK Recipe

Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do **swojego** repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało.

Podaj link do swojego repozytorium:

https://github.com/AG1F57T/ag1f57t.github.io/tree/main/LAB_H/ptw/lab-h

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. **Nie używaj LLM.**

Całkiem proste zadanie, dobrze wytłumaczone na labach co ułatwiło pracę

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.